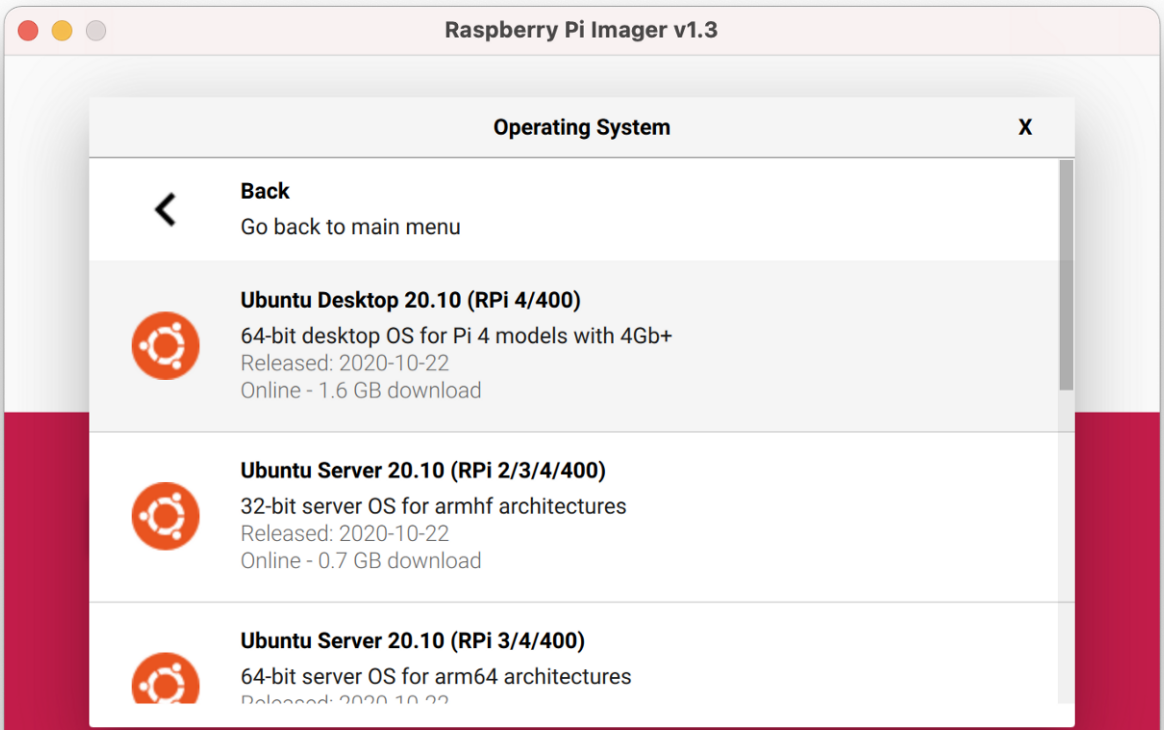
There are many reasons why you would want to put Ubuntu in your Raspberry Pi instead of the official Raspbian OS. In my case I usually use Raspberry Pi as a low-powered portable computer for small robotics and AI projects. Many drivers, packages and dependencies are only fully supported in Ubuntu (as ROS2 🤖 or OpenCV 👀) but not in Raspbian.

In this article I want to go through the different common steps you will need to successfully setup a Raspberry Pi 4 or Raspberry Pi 3 with the latest versions of Ubuntu for the ARM 64-bits architecture, currently **Ubuntu 20.10 Groovy Gorilla** and **Ubuntu 20.04 Focal Fossa**.

Step 1: Burn your Ubuntu OS image

First you will need to choose your Ubuntu version and edition. Depending on your requirements you will want to have a full desktop edition or just command line login. For a full desktop experience I recommend you at least a Raspberry Pi 4 with 4Gb of RAM and ***Ubuntu 20.10 Desktop*** edition. For a Raspberry Pi 3 I would probably stay with the ***Ubuntu Server 20.04 LTS*** and console access without graphical interface.

The most easy way to choose the latest compatible OS and burn the image into a SD card is to use the official Raspberry Pi Imager app. It includes an updated collection of compatible OS for every model of Raspberry Pi. You can go to the Ubuntu section and choose the version you need. Make sure you always choose a 64-bit version for desktop or server editions.



List of Ubuntu images for Raspberry Pi. Image by the author

For burning the image use a SD with at least 16GB. Because the Raspberry Pi will use the SD card not only for storage but also for expanding the memory available using the swap memory in the SD card is very important to use a fast SD card (the higher class the better). The *Raspberry Pi Imager* app will take care of all burning steps for you: format, write and validate image.

Boot the new OS and login for the first time to check everything seems alright. In Ubuntu default user login is ***ubuntu*** with password ***ubuntu***.

Step 2: Setup WiFi

The first task will be to setup Internet access via WiFi. Supposing you are currently offline the most straightforward way to do it is to edit directly the *netplan* configuration file:

```
$ sudo nano /etc/netplan/50-cloud-init.yaml
```

We need to add the wifi adaptor configuration. Type these lines after the ***ethernets:*** configuration and with the same indentation:

```
wifis:
    wlan0:
        optional: true
        access-points:
            “SSID-NAME-HERE”:
                password: “PASSWORD-HERE”
        dhcp4: true
```

Now you can apply the new configuration from *netplan* and check if there’s any error:

```
$ sudo netplan generate
$ sudo netplan apply
```

Check the *wlan0* interface has successfully connected and has an assigned IP:

```
$ ip addr show
# or just `ip a` or just `hostname -I` for a minimalistic approach
```

When Ubuntu OS connects to the Internet for the first time it will take some time to run updates in the background. Don't be surprised if *apt* or *apt-get* commands are locked for few minutes.

Step 3: Setup localhost name access

Having several machines in a wifi network with changing dynamic IPs makes very hard to connect to them every time. We want to be able to access every machine with his local hostname address. This works very well in Linux and Mac systems, for Windows you would need additional work like to install Bonjour for Windows. First we need to modify the hostname for your machine:

```
# Set host name to `ubuntu` (this is the default anyway)
$ sudo hostnamectl set-hostname ubuntu
```

Then we install the avahi daemon and enable the auto-discovery service:

```
# install avahi and enable daemon
$ sudo apt install -y avahi-daemon
```

Now your Ubuntu machine is accessible using ***ubuntu.local*** instead of a dynamic IP inside your network.

Step 4: Setup SSH

With local hostname and connectivity enabled, we want to be able to remotely login into the machine. Depending on your Ubuntu version SSH service could be activated by default. But just in case we try to install the SSH server package and allow traffic for it the integrated Ubuntu firewall:

```
$ sudo apt install openssh-server
# Allow Ubuntu firewall to accept ssh connections
```

```
$ sudo ufw allow ssh
```

If SSH is up you should be able to login from a remote machine using the default user name and hostname:

```
$ ssh ubuntu@ubuntu.local
```

Step 5: Setup swap memory size

Swap memory allows the OS to use storage space in the SD card as extra memory for running processes. This is very important to activate in the Raspberry Pi systems since power is very limited and we can out of memory very fast! First we need to install the configuration manager package (it may be installed already depending on your Ubuntu version):

```
$ sudo apt-get install zram-config
```

And now we can edit the swap memory setup file.

```
$ sudo nano /usr/bin/init-zram-swapping
```

You should look for a line where ***mem*** variable is defined and multiply the value by the Gigabytes in the SD card you want to assign to memory swapping. Usually 5GB is a good number:

```
# Inside /usr/bin/init-zram-swapping
# Multiply the available swap memory size. 5Gb should be good enough
mem=$(((totalmem / 2 / ${NRDEVICES}) * 1024 * 5))
```

Reboot the Raspberry Pi and check the new swap memory available with the ***htop*** command:

```
$ sudo reboot
. . .
```

```
$ htop
```

Step 7: Enable the Raspberry camera (optional)

You may want to activate your official Raspberry Pi camera. Usually in Raspbian you can use *raspi-config* to edit some firmware configuration. However in Ubuntu we can edit directly the firmware configuration file:

```
$ sudo nano /boot/firmware/config.txt
```

To activate the camera and increase the GPU memory size add these lines under ***[all]***:

```
start_x=1
gpu_mem=128
```

In a Raspberry Pi 4 you probably wouldn't need to modify the GPU memory size. Reboot and try to save a picture from your camera:

```
$ raspistill -o cam.jpg
```

Step 8: Enable i2c (optional)

In the same way with the camera you may want to have **i2c** communication activated, for example if you are using an extension hat. In Ubuntu **i2c** is usually activated by default but only for the root user. You would just need to add your user to the *i2c* group:

```
$ sudo usermod -aG i2c ubuntu
```

Logout and login again and check your new permission based on the **i2c** group:

```
$ i2cdetect -y 1
```

If you have permission to run it **i2c** have been successfully enable for your user.